FINAL PROJECT

Graph-Based Smart Contract Auditing

A graph attention network prototype, evaluated by leave-one-contract-out

[Student Name] · Student ID [Student ID] · October 2026

MOTIVATION

Deployed contracts cannot be patched

- Smart contracts hold real assets on Ethereum; a flaw is exploitable as soon as it is deployed.
- Three classes studied: **Reentrancy**, **Timestamp dependency**, **Arithmetic error**.
- Existing tools disagree with each other and flag many real contracts (Durieux et al., 2020).
- Idea: treat a contract as a graph of control flow and data flow, and learn over it.

RESEARCH QUESTIONS

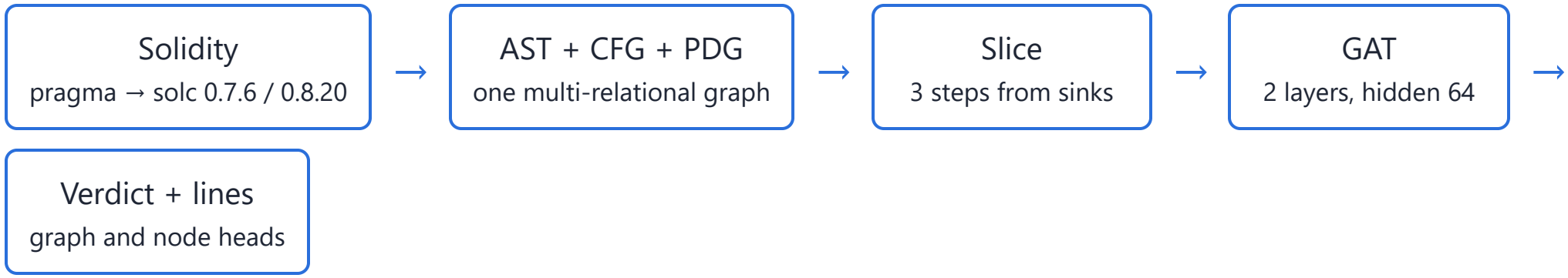
What this project asks

- **RQ1.** Can a GAT over a sliced contract graph separate vulnerable from safe contracts?
- **RQ2.** Does it generalize to a contract it has not seen, compared with simple baselines?
- **RQ3.** Can its node scores point to the vulnerable line?

> We report a feasibility result, not a detection rate.

PIPELINE

From source to verdict



Sinks: external calls, `block.timestamp`, arithmetic operators. Thresholds: 0.5 contract, 0.3 node (set by hand).

DATA AND PROTOCOL

Six demo contracts, leave one out

- One vulnerable and one safe contract per class (hand-written, MIT).
- Each held-out contract: retrain on the other five, then decide for all three classes.
- 18 decisions per method, of which **3** are positive.
- GAT: 5 seeds. Baselines: sink-presence rule and logistic regression.
- **Sink rule:** flag a class if any sink for it exists. No training.
- **Logistic regression:** pooled node features, no message passing.

RESULTS

Leave-one-contract-out, 18 decisions

Method	Precision	Recall	F1	Accuracy
GAT (mean of 5 seeds)	0.000	0.000	0.000	0.767 ± 0.022
Sink-presence rule	0.429	1.000	0.600	0.778
Logistic regression	0.000	0.000	0.000	0.833

> The GAT flags none of the three held-out vulnerable contracts, in any seed.

IN-SAMPLE VS. HELD OUT

The fit does not transfer

18 / 18

correct decisions when trained and tested on the same six contracts (F1 1.000)

0 / 3

held-out vulnerable contracts detected under leave-one-contract-out (recall 0.000)

> With one positive per class, holding out a contract removes the only example of its pattern.

FIGURE

Method comparison



ERROR ANALYSIS

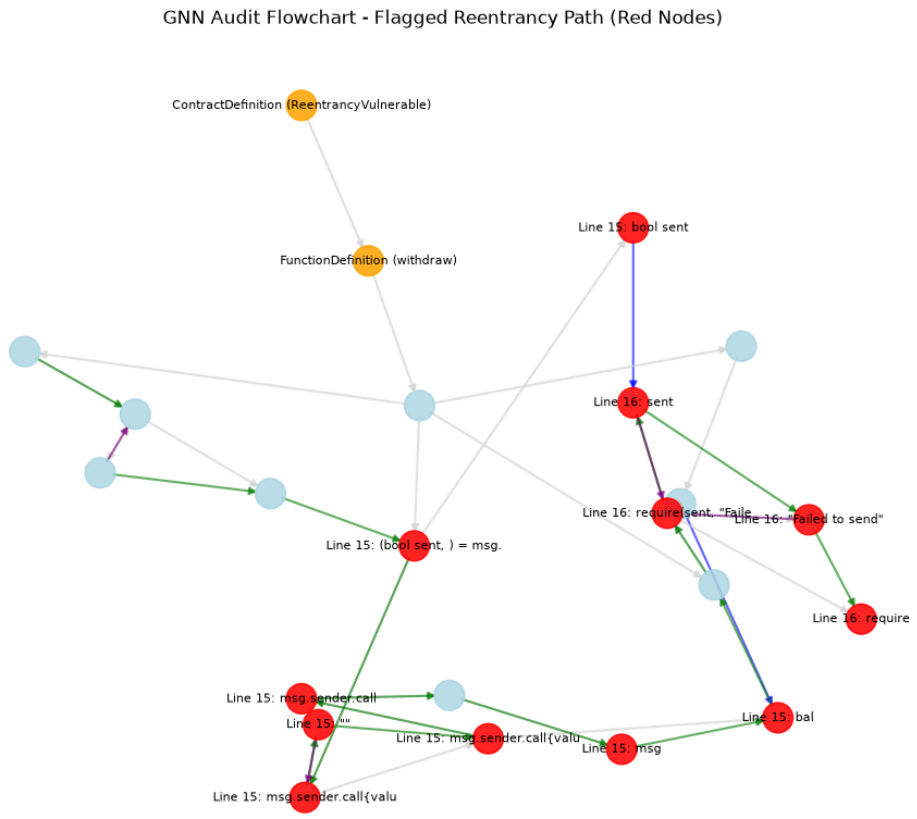
Why each error happens

- **GAT:** errors repeat in every seed. Held-out Reentrancy: the only training example is the safe contract, which scores the pattern as safe. Held-out safe contract scored 0.96 for Reentrancy.
- **Sink rule:** `send` matches inside `msg.sender` and `"Failed to send"`, so two Arithmetic contracts are flagged for Reentrancy.
- **Sink rule:** flags any `*`, so it cannot separate checked and unchecked arithmetic.

LOCALIZATION

Lines: in-sample yes, held out no

- In-sample, Reentrancy is localized to lines 15 and 16 (external call and its check).
- In-sample, Timestamp is localized to line 9, after a label fix.
- Arithmetic is flagged at 98.25% but not localized.
- Held out: localization fails in all 15 vulnerable-contract evaluations.



DEFECTS FOUND AND FIXED

Found while checking the work

- **Wrong training labels** for two contracts (lines 10 and 14 instead of 9 and 12).
- **Legacy-AST operator lookup** failed for solc 0.7.6, so the arithmetic contract had no sink.
- **Unreproducible numbers** in the earlier draft removed; training is seeded and logged.
- **Unverifiable references** removed; the rest checked against publisher or arXiv pages.

> Pre-fix logs are kept in `results/logs/before_fixes/`. Full list: `FAILURE_LOG.md`.

LIMITATIONS

What this does not show

- Six contracts: precision and recall are not estimates of real performance.
- Keyword features match substrings (`now`, `send`).
- Thresholds set by hand; compiler versions 0.4–0.6 untested.
- Public benchmarks (SmartBugs, SolidiFI) not yet used.

NEXT STEPS

What would make a detection claim possible

1. Evaluate on SolidiFI and SmartBugs with a held-out split.
2. Report precision, recall and F1 with confidence intervals.
3. Compare with Oyente, SmartCheck and Slither on the same contracts.
4. Run an ablation for the keyword features.

EVIDENCE

Everything is in the repository

- Code, saved weights, three executed notebooks, all metrics and logs.
- `results/metrics.json` holds every number in the manuscript and slides.
- Data card, model card, ethics, AI-use disclosure, failure log, reproducibility checklist.
- Reproduce: `python evaluate.py` (see `README.md`).